

手写数字识别

1. 实验目的

- 理解卷积神经网络的基本原理与结构
- 熟悉基于 PyTorch 的模型构建、训练与评估流程
- 掌握手写数字识别任务中的数据处理与调参方法

2. 实验环境

2.1 软件环境

- python3.8
- torch $\geq$ 2.0.0
- torchvision $\geq$ 0.15.0
- matplotlib $\geq$ 3.7.0

2.2 实验数据集-MNIST数据集

- MNIST 是经典手写数字识别数据集。
- 训练集：60,000 张 28 $\times$ 28 灰度图像。
- 测试集：10,000 张 28 $\times$ 28 灰度图像。 - 分类任务：10 类（数字 0-9）。

3. 实验原理

3.1 卷积神经网络（CNN）的基本结构

- 卷积层（Convolutional Layer）：通过卷积操作提取图像的局部特征
- 池化层（Pooling Layer）：通过下采样操作减少特征图的
- 激活函数（Activation Function）：引入非线性变换，增强网络表达能力，常用的有ReLU、Sigmoid等
- 全连接层（Fully Connected Layer）：将提取的特征映射到输出类别空间

3.2 卷积神经网络的网络结构

- 本实验使用的 CNN 模型结构如下：

```
├─ 输入层
│   └─ 28×28×1 图像
├─ 第一卷积块
│   ├── Conv2d: 1→16 通道, 5×5 卷积核, padding=2
│   ├── ReLU 激活函数
│   └─ MaxPool2d: 2×2 池化 → 14×14×16
├─ 第二卷积块
│   ├── Conv2d: 16→32 通道, 5×5 卷积核, padding=2
│   ├── ReLU 激活函数
│   └─ MaxPool2d: 2×2 池化 → 7×7×32
├─ 展平层
│   └─ 32×7×7 = 1568 维特征向量
└─ 全连接层
    └─ Linear: 1568 → 10 输出 (Softmax 分类)
```

3.3 核心代码实现

数据加载与预处理:

```
def build_mnist_loaders(batch_size=BATCH_SIZE, data_root=DATA_ROOT):
    transform = build_transform(with_normalize=True)

    train_data = datasets.MNIST(
        root=data_root,
        train=True,
        transform=transform,
        download=True,
    )
    test_data = datasets.MNIST(
        root=data_root,
        train=False,
        transform=transform,
        download=True,
    )

    train_loader = DataLoader(dataset=train_data, batch_size=batch_size,
```

```
shuffle=True)
    test_loader = DataLoader(dataset=test_data, batch_size=batch_size,
                             shuffle=False)
    return train_loader, test_loader
```

模型结构与前向传播:

```
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Sequential(
            nn.Conv2d(
                in_channels=1,
                out_channels=16,
                kernel_size=5,
                stride=1,
                padding=2,
            ),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2),
        )
        self.conv2 = nn.Sequential(
            nn.Conv2d(
                in_channels=16,
                out_channels=32,
                kernel_size=5,
                stride=1,
                padding=2,
            ),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2),
        )
        self.out = nn.Linear(32 * 7 * 7, 10)

    def forward(self, x):
        x = self.conv1(x)
        x = self.conv2(x)
        x = x.view(x.size(0), -1)
        return self.out(x)
```

训练与评估循环:

```
for step, (x, y) in enumerate(train_loader):
    x = x.to(device)
    y = y.to(device)

    output = model(x)
    loss = loss_func(output, y)
    pred = torch.argmax(output, dim=1)
```

```
train_correct += (pred == y).sum().item()
train_total += y.size(0)

optimizer.zero_grad()
loss.backward()
optimizer.step()

if step % 50 == 0:
    train_accuracy = train_correct / train_total if train_total > 0 else 0.0
    test_accuracy = evaluate(model, test_loader, device)
    print(
        f"Epoch: {epoch}, Step: {step}, "
        f"Loss: {loss.item():.4f}, "
        f"TrainAcc: {train_accuracy:.4f}, TestAcc: {test_accuracy:.4f}"
    )

    writer.writerow(
        [
            epoch,
            step,
            f"{loss.item():.6f}",
            f"{train_accuracy:.6f}",
            f"{test_accuracy:.6f}",
        ]
    )
```

4. 训练核心流程

4.1 训练流程概述

- 数据加载与预处理
- 模型定义与初始化
- 训练循环：前向传播 → 计算损失 → 反向传播 → 参数更新
- 评估与日志记录

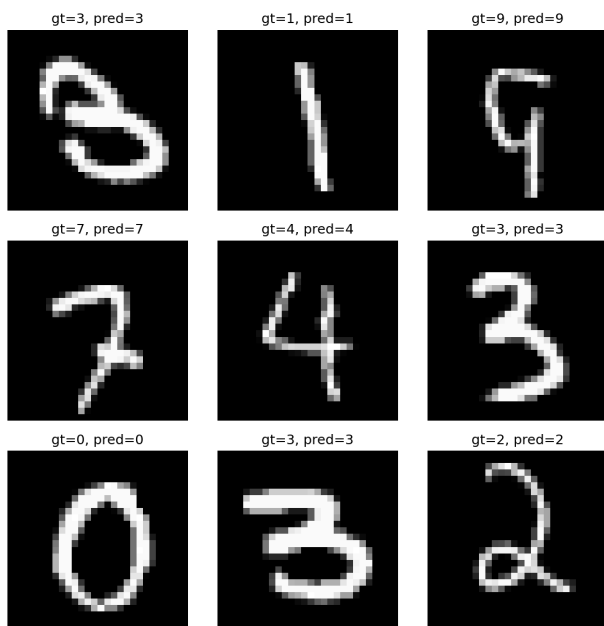
4.2 训练配置

- 学习率 (Learning Rate): 0.001
- 批大小 (Batch Size): 64
- 训练轮数 (Epochs): 10

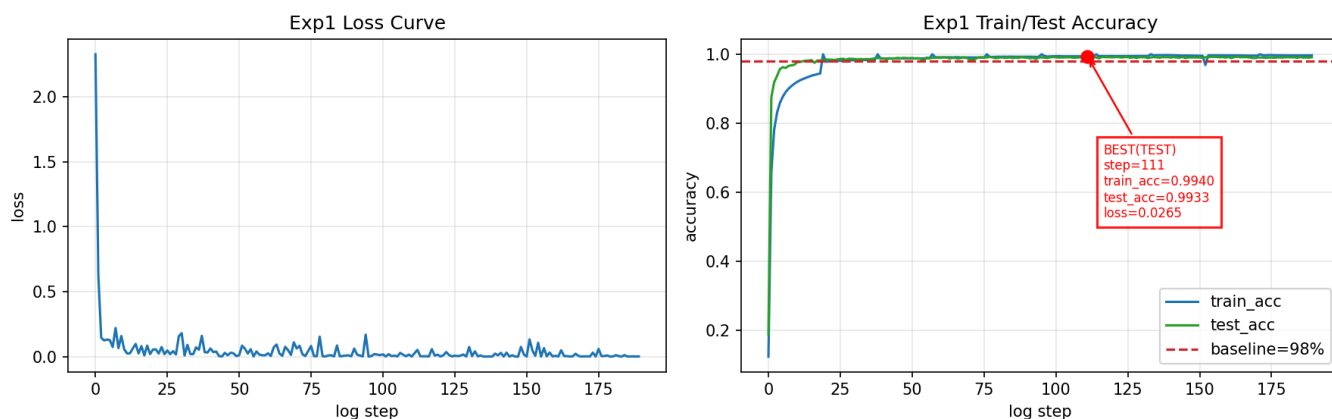
5. 实验总结

4. 实验结果

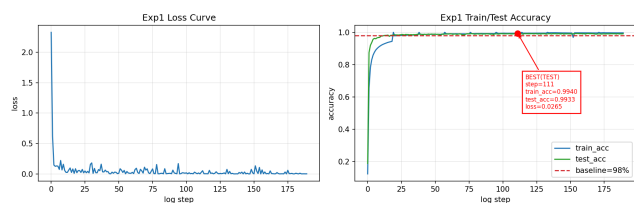
- 如图所示，模型能够精准的识别出测试集中的手写数字，具有较好的泛化能力。



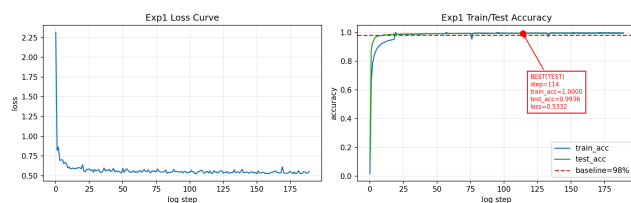
- 在默认配置（adam优化器，ce损失函数，学习率0.001，训练10轮）下，模型在测试集上的准确率达到99.3%，表现良好。



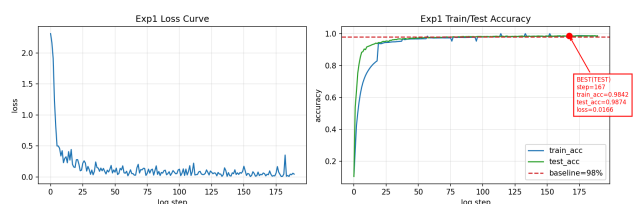
- 在不同的模型配置下，模型的性能表现如下：



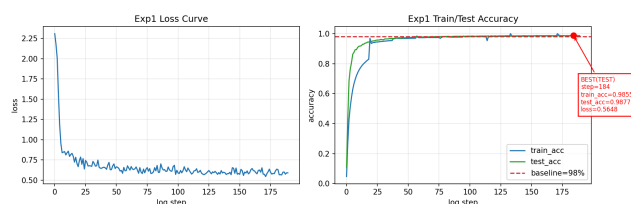
(Adam + CE)



(Adam + CE_LS)



(SGD + CE)



(SGD + CE_LS)

- 可见adam优化器收敛速度与最终正确率均优于sgd，而损失函数对于结果的影响不大。

5. 实验总结

- 本实验基于 MNIST 数据集完成了手写数字识别任务，使用两层卷积块加全连接层的 CNN 结构，实现了从数据预处理、模型训练到结果可视化的完整流程。实验结果表明，该模型能够较准确地识别测试集中的数字，在默认配置（Adam + CE，学习率 0.001，训练 10 轮）下测试准确率达到 99.3%，说明所构建的网络具备较好的特征提取能力与泛化性能。
- 对比不同优化器与损失配置后可见，Adam 的收敛速度和最终效果整体优于 SGD；在本实验设置下，损失函数的选择对最终性能影响相对有限。总体而言，CNN 在 MNIST 这类标准视觉分类任务上表现稳定、实现成本低，适合作为图像分类入门模型。